MedTech
Mediterranean
Institute of Technology

CS202 – ADVANCED PROGRAMMING

SERIALIZATION IN JAVA

SOPHOMORE SE

SPRING 2026

Agenda

1. Serialization
2. Serialization in Java
3. Binary serialization
4. XML serialization

Serialization – definition

Java provides a mechanism, called **object serialization** where an object can be represented as a **sequence of bytes** (or stream) that includes the **object's data** as well as information about the **object's type** and the **data types stored in it**.

Deserialization is the **reverse process** where the byte stream is used to **recreate the actual Java object in memory**. This mechanism is used to persist the object.

After a serialized object has been written into a file, it can be read from that file and then deserialized, ie. the type information and bytes represented by that object and then, its data can be used to recreate the object in memory.

Serialization – objectives

This process consists of storing an object in another form **outside the JVM** aiming to

- Ensuring the data **persistence**
  - We can store an object and then **restore** it for another runtime task
  - We can store objects in a mass memory (to be retereived later)
  - Serialized objects can be stored using **files** or **databases**
- To allow object **transmission**
  - For example, exchanging objects between processes (locally or remotely via networks)
  - From one application to another
  - An application performs calculus and makes it available for other applications
  - Configuration files

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Serialization – languages

Framework/languages examples allowing serialization process :

- .NET (Microsoft)
- C++
- OCaml
- Python,
- Java, …

synonymous

- Marshalling / Unmarshalling
- Sometimes linearization

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Serialization in Java

Which objects can be serialized in Java?

- Objects implanting the interface **Serializable**
- This interface est « empty », it does not contain implemented methods
- However, an object that does not implement Serializable cannot be serialized

thus, attributes instantiating other classes must also implement the same interface Serializable

It is intersting to serialize meaningful objects

For example, it does not make a sens to serialize an **InputStream**

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Serialization in Java - characteristics

Simple to use/do : everything is done automatically by introspection

- Serialize a graph of objects : all the objects of this graph implement the interface **Serializable**
  - Most of the objects (arrays, lists, dictionaries ...) of the standard API are already Serializable
- All the attributes of an object will be serialized (even those are **private**) except those declared as **transient**
- The information stored in transient attributes, will not saved/considered during the serialization task
- In the **deserialization** task, the attributes contain the null value: risk of *NullPointerException*
- The used **class version** for **serialization** should be the same to be used for **deserialization**

```
private static final long serialVersionUID = 4242L;
// to be defined explicitely … otherwise the compiler generates it automatically
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

Serialization – How does it works? (1/2)

The object to be serialized is **decomposed** into small and basic/atomic elements. Each of them will be **encoded**

These serialized objects form a **graph**

- For example, to remeber that the attribute of two different objects contain the same object

The **attributes** of an object should be **serializable**

The **structure** of an object must be kept to reconstruct the object during the **deserialization** task (Example : arrays, lists , etc.)

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Serialization – How does it works? (2/2)

To perform this mechanism in Java, we need the interface **Serializable**, as well as the classes **ObjectOutputStream** and **ObjectInputStream**.

The interface **Externalizable** may be used to implement a customized serialization.

Serializable is an interface allowing us to identify the serialized classes (or to be serialized).

Thus, to serialize an object of a given class, it should implement **Serializable** or inherits from another serializable class.

A serializable attribute should meet te following conditions:

Should have a primitive data type, or a serializable type (defined by the programmer).

Should not be **static**.

Should not be **transient**.

Should not be inherited from a non serializable mother class.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Example

```
// creating a Serializable class
 public  class user implements Serializable {

    // attributes
    private String fname, lname ;

    // prohibit the serialization of the following attribute
    private  transient Connection con ;
}
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Serialization – Storage (1/2)

There are several formats:

- Format XML (eXtended Markup Language)
  - Readable : <description> text </description> → **XMLEncoder** et **XMLDecoder**
  - Binary derived format: Binary XML
- Format JSON (JavaScript Object Notation) derived from JavaScript (binary derived format: BSON)
- Format YAML (YAML Ain't Markup Language)

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Serialisation – Storage (2/2)

- XRD Format (External Data Representation)
- CSV Format (Comma Separated Values) :
- Format INI : textual format used for configuration and also for properties files in java
- Binary Format

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Supported formats

- Specifying generic formats using schema (such XML) considering some constraints such (DTD, XSD, RelaxNG, Schematron, ...)
- Data exchange between heterogenous applications to check the data coherence
- Some XML languages derived from shema (XHTML, MathML, OpenDocument, SVG...)

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

API Reading/writning approaches

Handling trees/graphs of objects

- Easiest approach : Direct and random access to structures in memory
- Expensive approach : Structures are fully in memory

Handling events

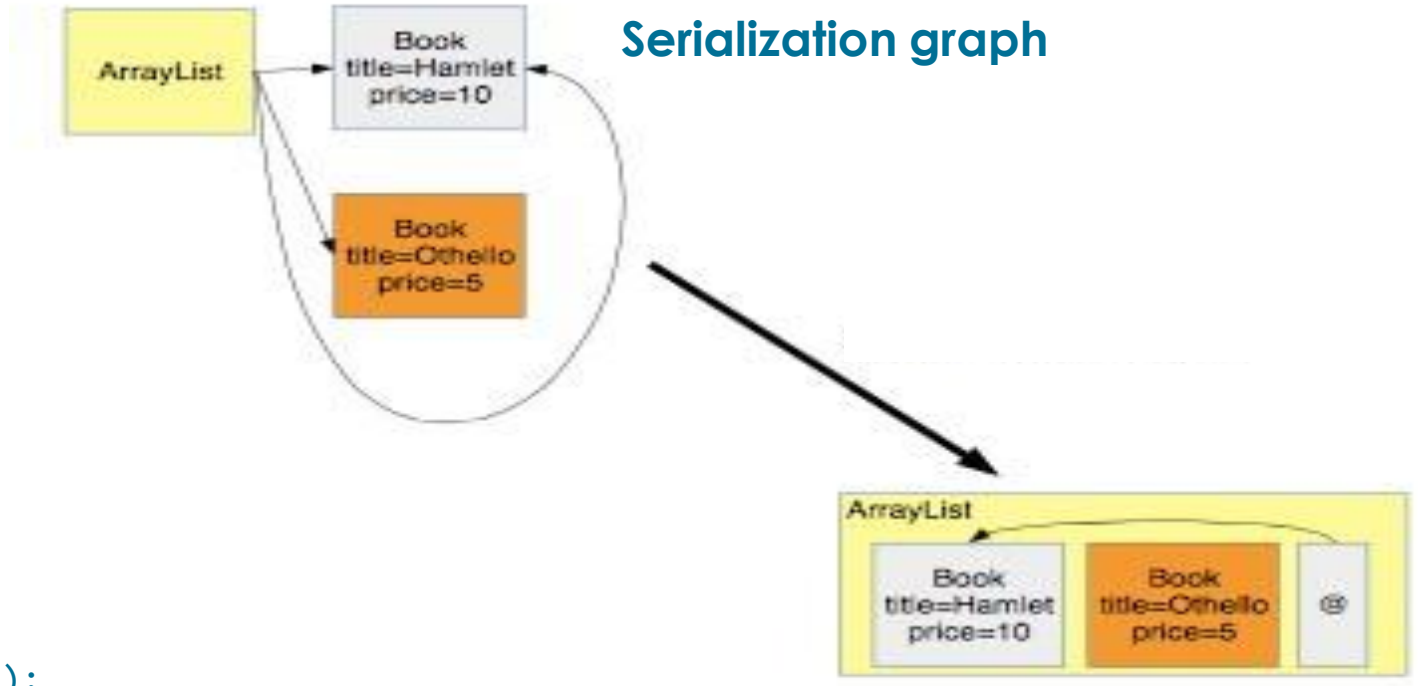
- Opening and Reading objects / arrays, writning values, closing objects
- Structures outside memory, serial access
- Styles of reading API :
  - *Pushing* reminding ***callback*** functions
  - *Pulling* by **event iterators**

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Example 2 - Serialization

```
import java.io.*;
import java.util.*;
public class Book implements Serializable
{        private final String title;
         private final int price;
         public Book(String title, int price)
         {        this.title = title;
                  this.price = price;}

         public static void main(String[] args)
         {
                  List<Book> l = new ArrayList<Book>();
                  l.add(new Book("Hamlet", 10));
                  Book b = new Book("Othello", 5);
                  l.add(b);
                  l.add(b);
                  try {
                           ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(file));
                           oos.writeObject(l);
                  }catch(IOException e) {System.out.println(e);}
         }
}
```



Example 2 – Reconstitution of serialized objects

```
List<Book> l = null;
try {
    ObjectInputStream ois = new ObjectInputStream(new FileInputStream("serializedBooks.txt"));
    l = ois.readObject();
} catch (Exception e) {}
```

Exceptions:

- **FileNotFoundException** → when the file is not available (found)
- **ClassNotFoundException** → when the class is not found, defined, etc.
- **InvalidClassException** → problem with the class versions
- **ObjectStreamException** → distorted file
- Check serialized objects thanks to **ObjectInputValidation** → the method *validateObject()* checks the object and may throw an **InvalidObjectException** in case of problems

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

Managing classes versions

- Each class implementing Serializable has a version number
- This number is stored in long field named **serialVersionUID**
- The user can specify the value of the version number

```
private static final long serialVersionUID = 1 ;
```

Otherwise, it is initialized automaticalyy by the compiler

This value is always assigned when the serialization is performed (only in this case)

It allows to verify if the class has changed during serialization and deserialization processes

In case of version incompatibility, an exception occurs during the deserialization process

This number is handled in the main method and should be updated when the class is modified

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Example

```
import java.io.*;
public class Person implements Serializable {
        private String name;
        int age;
        private static final long serialVersionUID=1;
        public Person (String _name, int _age)
        {       this.name = _name;
                this.age = _age; }
        public String toString(){
                return (name +", "+age);
        }
}
```

Object storage using files in Java

Declaring the objects to be serialized as implementing the interface Serializable

Then, the objects can be written as output stream objects

- Class **ObjectOutputStream**
  - Methode *void writeObject(Object o)*
  - If the object to be serialized or the objects defined as attributes are not serializable, anexception occurs

Reading objects

- Class **ObjectInputStream**
  - Method *Object readObject()*
  - We should add a casting operation for the retreived objects

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Example of binary serialization

```
import java.io.*;
public class Person implements Serializable {
        private String name;
        private Dog d;

        public Person (String _name, Dog _dog){
                this.name=_name;
                this.d=_dog; }

        public String toString(){
                return (name +", " +d.toString()); }

        public Dog getDog(){
                return d; }

}
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Example

```
import java.io.*;
public class Dog implements Serializable{
        private String name;

        public Dog (String _name){
                this.name=_name; }

        public String toString(){
                return name; }
}
```

Exemple

```
import java.io.*;
public class SerializedDogs{
        public static void main(String[]args){
                try{
                Dog d1=new Dog("Laika");
                Person p1=new Person ("Alice”,d1)
                Person p2=new Person ("Bob”,d1);
                Person p3=new Person ("Charles”,d1);
                FileOutputStream fo = new FileOutputStream ("PersonObj.bin");
                ObjectOutputStream os=new ObjectOutputStream(fo);
                os.writeObject(p1);
                os.writeObject(p2);
                os.writeObject(p3);
                os.close();
                fo.close();
                }catch(Exception e){
                        e.printStackTrace();}
} }
```

Example

```
import java.io.*;
public class DeserializedDogs{
        public static void main(String[]args){
                try{
                int nbP=3;
                Person p = null;
                FileInputStream fi = new FileInputStream("PersonObj.bin");
                ObjectInputStream ois = new ObjectInputStream(fi);

                while(nbP>0){
                        p=(Person)ois.readObject();
                        System.out.println(p.toString());
                        nbP—; }
                ois.close();
                fi.close(); }
                catch(Exception e)
                        { e.printStackTrace(); } } }
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

XML Serialization

The XML serialization is a persistence mechanism provided by Java 1.4 thanks to the classes

- **XMLEncoder**
- **XMLDecoder**

XML Serialization

```
Public class DVD {

        private List movies = new ArrayList();

        public DVD(){}

        public List getMovies() {
                return movies; }

        public void setMovies(List movies) {
                this.movies = movies; }

        public String toString(){
                String movies="";
                for(Object movie:getMovies()){
                    movies += ((Movie) movie).getName()+", "; }
                return movies; }
}
```

XML Serialization

```
public class Movie {
    private String name;
    private int runtime;
    private String directors;
    private int released;
    private String cast;
    public Movie(){}

    public Movie(String name, int runtime, String
    directors,int released, String cast) {

         this.name = name;
         this.runtime = runtime;
         this.directors = directors;
         this.released = released;
         this.cast = cast;}

    public String getName() {
         return name;}

    public void setName(String name) {
         this.name = name;}

    public int getRuntime() {
         return runtime;}

    public void setRuntime(int runtime) {
         this.runtime = runtime;}

    public String getDirectors() {
         return directors;}

    public void setDirectors(String directors) {
         this.directors = directors;}

    public int getReleased() {
         return released;}

    public void setReleased(int released) {
         this.released = released;}

    public String getCast() {
         return cast;}

    public void setCast(String cast) {
         this.cast = cast;
    }

}
```

```
<?xml version="1.0" encoding="UTF--8"?>
<java version="1.8.0_151" class="java.beans.XMLDecoder">
 <object class="Serialization.DVD" id="DVD0">
  <void property="movies">
   <void method="add">
    <object class="Serialization.Movie">
     <void property="cast">
      <string>Matt Damon, Franka Potente</string>
     </void>
     <void property="directors">
      <string>Doug Liman</string>
     </void>
     <void property="name">
      <string>The Bourne Identity</string>
     </void>
     <void property="released">
      <int>2002</int>
     </void>
     <void property="runtime">
      <int>119</int>
     </void>
    </object>
   </void>
   <void method="add">
    <object class="Serialization.Movie">
     <void property="cast">
      <string>Matt Damon, Franka Potente, Joan Allen</string>
     </void>
     <void property="directors">
      <string>Paul Greengrass</string>
     </void>
     <void property="name">
      <string>The Bourne Supremacy</string>
     </void>
     <void property="released">
      <int>2004</int>
     </void>
     <void property="runtime">
      <int>108</int>
     </void>
    </object>
   </void>
   <void method="add">
    <object class="Serialization.Movie">
   <void property="cast">
      <string>Matt Damon, Edgar Ramirez, Joan Allen</string>
     </void>
     <void property="directors">
      <string>Paul Greengrass</string>
     </void>
     <void property="name">
      <string>The Bourne Ultimatum</string>
     </void>
     <void property="released">
      <int>2007</int>
     </void>
     <void property="runtime">
      <int>115</int>
     </void>
    </object>
   </void>
   <void method="add">
    <object class="Serialization.Movie">
     <void property="cast">
      <string>Jeremy Renner, Rachel Weisz, Edward Norton</string>
     </void>
     <void property="directors">
      <string>Tony Gilroy</string>
     </void>
     <void property="name">
      <string>The Bourne Legacy</string>
     </void>
     <void property="released">
      <int>2012</int>
     </void>
     <void property="runtime">
      <int>135</int>
     </void>
    </object>
   </void>
  </void>
 </object>
</java>
```

```
public class DeserializeFromXML {

        private static final String SERIALIZED_FILE_NAME="dvd.xml";

        public static void main(String[] args) {
                XMLDecoder decoder=null;
                try {
                        decoder=new XMLDecoder(new BufferedInputStream(new
                           FileInputStream(SERIALIZED_FILE_NAME)));
                } catch (FileNotFoundException e) {
                         System.out.println("ERROR: File dvd.xml not found");}
                DVD bourneSeries=(DVD)decoder.readObject();
                System.out.println(bourneSeries);

        }
}
```

How can subclass avoid Serialization if its superClass has implemented Serialization interface in java

- Not possible, as subclass always inherits all features from its parent class
- define writeObject() method and you are done
- define writeObject() method and throw NotSerException()
- define writeObject() method and throw NotSerializableException()

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues